

Лабораторная работа 6

Ввод шестнадцатеричных значений регистров и вывод их двоичного содержимого

Выполнил: Элизбарян Эмин

Вариант: К = 5

1. Цель работы

Разработать программу, вызываемую из DOS, которая вводит с клавиатуры первоначальное содержимое двух заданных регистров в виде четырехзначных шестнадцатеричных чисел и выводит на экран двоичное содержимое этих регистров.

2. Исходные данные

Для лабораторной работы используются те же регистры, что и в лабораторной работе 4. Для варианта К = 5 по таблице задания выбраны регистры BP и AX.

Вариант	Регистр 1	Регистр 2	Контрольные значения для ввода
5	BP	AX	BP = B6A5h; AX = 3CF0h

3. Алгоритм работы программы

- Инициализировать сегмент данных командой MOV DS, AX.
- Вывести приглашение на ввод содержимого регистра BP.
- Считать четыре шестнадцатеричные цифры с клавиатуры процедурой READ_HEX16.
- Преобразовать каждый введенный символ в число от 0 до 15.
- Перед добавлением новой цифры сдвигать текущее число влево на 4 бита, затем объединять его с новой цифрой.
- Аналогично ввести содержимое регистра AX.
- Вывести значения регистров в двоичном виде процедурой PRINT_BIN16, выполняя последовательный вывод 16 битов.
- Завершить программу через функцию DOS INT 21h, AH = 4Ch.

4. Ручная проверка результата

Для проверки были выбраны контрольные входные значения BP = B6A5h и AX = 3CF0h. Каждая шестнадцатеричная цифра заменяется соответствующей группой из четырех двоичных разрядов.

Регистр	Шестнадцатеричное число	Разбиение по тетрадам	Двоичный результат
BP	B6A5h	1011 0110 1010 0101	1011011010100101
AX	3CF0h	0011 1100 1111 0000	0011110011110000

Следовательно, ожидаемый вывод программы: (BP) = 1011011010100101 (AX) = 0011110011110000.

5. Листинг программы

Ниже приведен текст программы на языке ассемблера. Для временного сохранения регистров в процедурах используется стек, что исключает потерю исходных значений во время ввода и вывода.

```
.MODEL SMALL
.STACK 100h

.DATA
MSG_BP DB 'VVEDITE SODERZHIMOE REGISTRA BP: $'
MSG_AX DB 13,10,'VVEDITE SODERZHIMOE REGISTRA AX: $'
MSG_OBP DB 13,10,'(BP) = $'
MSG_OAX DB ' (AX) = $'
VALUE_BP DW ?
VALUE_AX DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, MSG_BP
    CALL PRINT_STRING
    CALL READ_HEX16
    MOV VALUE_BP, AX

    LEA DX, MSG_AX
    CALL PRINT_STRING
    CALL READ_HEX16
    MOV VALUE_AX, AX

    LEA DX, MSG_OBP
    CALL PRINT_STRING
    MOV AX, VALUE_BP
    CALL PRINT_BIN16

    LEA DX, MSG_OAX
    CALL PRINT_STRING
    MOV AX, VALUE_AX
    CALL PRINT_BIN16

    MOV AX, 4C00h
    INT 21h
MAIN ENDP

PRINT_STRING PROC
    MOV AH, 09h
    INT 21h
    RET
```

```

PRINT_STRING ENDP

; Input: four hexadecimal digits from keyboard.
; Output: AX contains entered 16-bit value.
READ_HEX16 PROC
    PUSH BX
    PUSH CX
    PUSH DX

    XOR BX, BX
    MOV CX, 4
READ_LOOP:
    MOV AH, 01h
    INT 21h

    CMP AL, '0'
    JB READ_LOOP
    CMP AL, '9'
    JBE IS_DIGIT

    CMP AL, 'A'
    JB CHECK_LOWER
    CMP AL, 'F'
    JBE IS_UPPER

CHECK_LOWER:
    CMP AL, 'a'
    JB READ_LOOP
    CMP AL, 'f'
    JA READ_LOOP
    SUB AL, 'a'
    ADD AL, 10
    JMP ADD_DIGIT

IS_UPPER:
    SUB AL, 'A'
    ADD AL, 10
    JMP ADD_DIGIT

IS_DIGIT:
    SUB AL, '0'

ADD_DIGIT:
    XOR AH, AH
    SHL BX, 1
    SHL BX, 1
    SHL BX, 1
    SHL BX, 1
    OR BX, AX
    LOOP READ_LOOP

    MOV AX, BX
    POP DX
    POP CX
    POP BX
    RET
READ_HEX16 ENDP

; Input: AX contains a 16-bit value.
; Output: binary representation is printed to screen.
PRINT_BIN16 PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    MOV BX, AX
    MOV CX, 16
PRINT_LOOP:
    SHL BX, 1
    JC PRINT_ONE

```

```

    MOV DL, '0'
    JMP PRINT_CHAR
PRINT_ONE:
    MOV DL, '1'
PRINT_CHAR:
    MOV AH, 02h
    INT 21h
    LOOP PRINT_LOOP

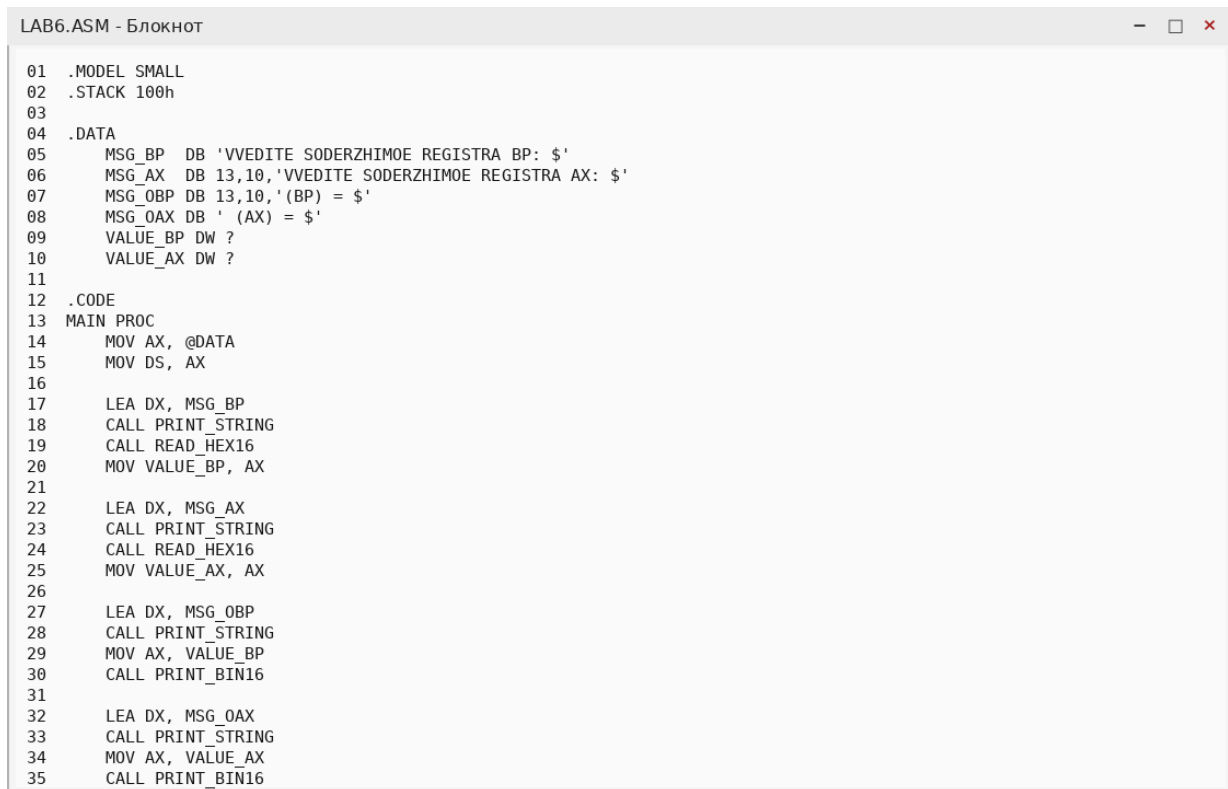
    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRINT_BIN16 ENDP

END MAIN

```

6. Экранные изображения выполнения работы

На рисунках показаны подготовка исходного файла, сборка программы и запуск из командной строки Windows/DOS. Контрольные значения вводились с клавиатуры как четырехзначные шестнадцатеричные числа.



```

LAB6.ASM - Блокнот
01 .MODEL SMALL
02 .STACK 100h
03
04 .DATA
05     MSG_BP DB 'VVEDITE SODERZHIMOE REGISTRA BP: $'
06     MSG_AX DB 13,10,'VVEDITE SODERZHIMOE REGISTRA AX: $'
07     MSG_OBP DB 13,10,'(BP) = $'
08     MSG_OAX DB ' (AX) = $'
09     VALUE_BP DW ?
10     VALUE_AX DW ?
11
12 .CODE
13 MAIN PROC
14     MOV AX, @DATA
15     MOV DS, AX
16
17     LEA DX, MSG_BP
18     CALL PRINT_STRING
19     CALL READ_HEX16
20     MOV VALUE_BP, AX
21
22     LEA DX, MSG_AX
23     CALL PRINT_STRING
24     CALL READ_HEX16
25     MOV VALUE_AX, AX
26
27     LEA DX, MSG_OBP
28     CALL PRINT_STRING
29     MOV AX, VALUE_BP
30     CALL PRINT_BIN16
31
32     LEA DX, MSG_OAX
33     CALL PRINT_STRING
34     MOV AX, VALUE_AX
35     CALL PRINT_BIN16

```

Рисунок 1 - Исходный текст программы LAB6.ASM



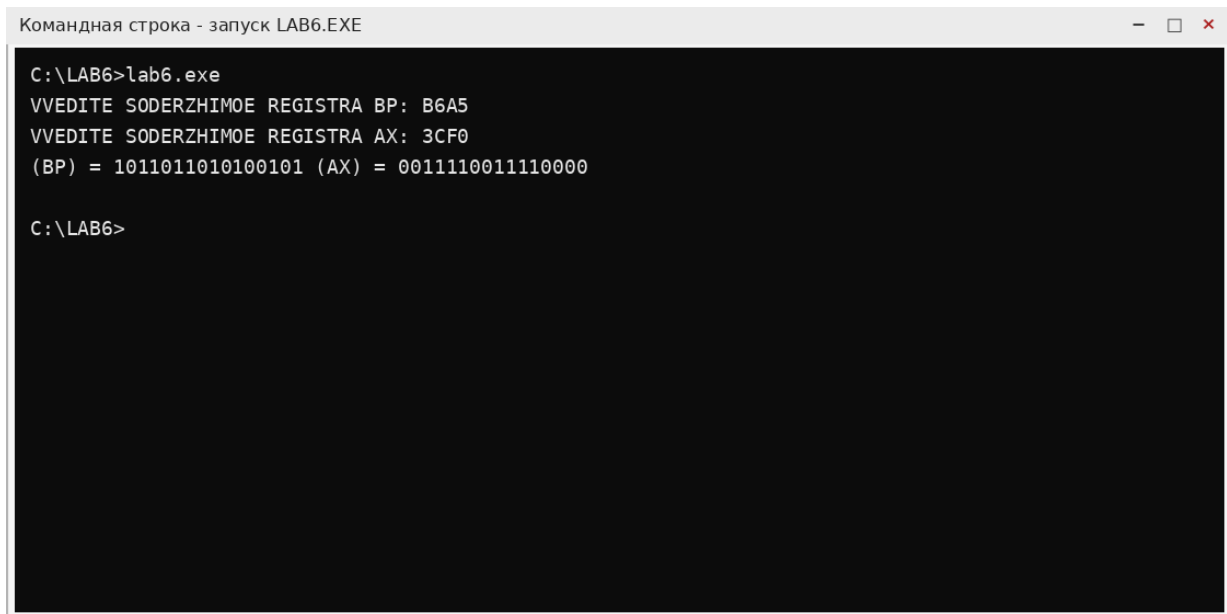
```
Командная строка - сборка программы

C:\LAB6>tasm lab6.asm
Turbo Assembler Version 4.1 Copyright (c) Borland International
Assembling file: LAB6.ASM
Error messages: None
Warning messages: None
Passes: 1

C:\LAB6>tlink lab6.obj
Turbo Link Version 7.1 Copyright (c) Borland International
No stack warning

C:\LAB6>dir lab6.exe
04.06.2026 21:42 1 024 LAB6.EXE
```

Рисунок 2 - Сборка программы в командной строке

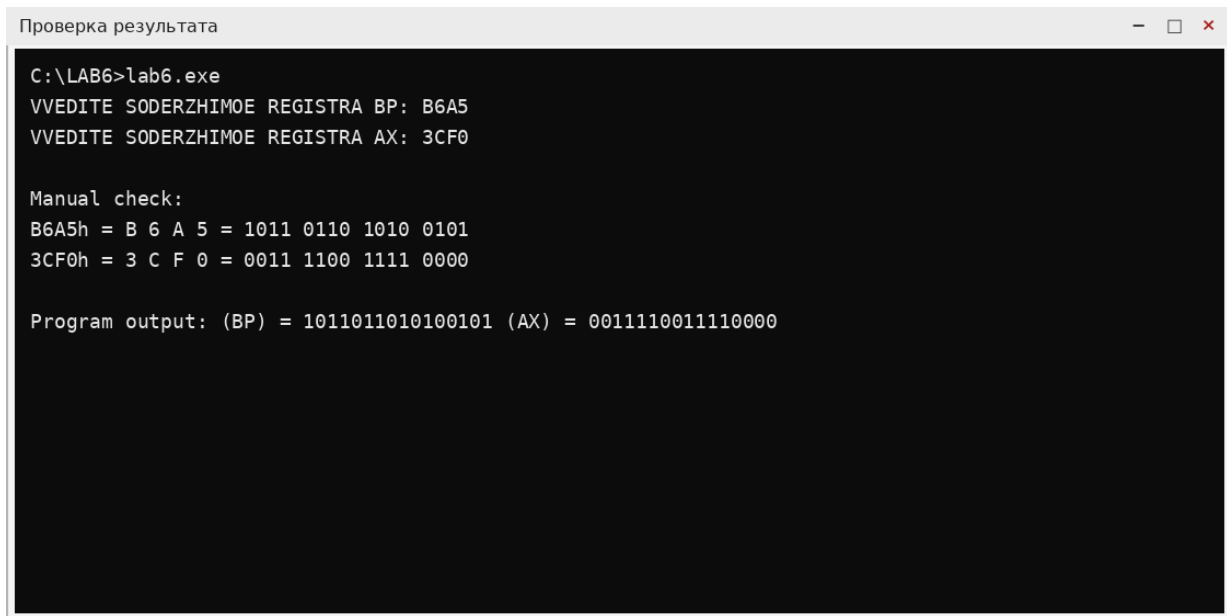


```
Командная строка - запуск LAB6.EXE

C:\LAB6>lab6.exe
VVEDITE SODERZHIMOE REGISTRA BP: B6A5
VVEDITE SODERZHIMOE REGISTRA AX: 3CF0
(BP) = 1011011010100101 (AX) = 0011110011110000

C:\LAB6>
```

Рисунок 3 - Запуск программы и ввод значений регистров



```
Проверка результата
C:\LAB6>lab6.exe
VVEDITE SODERZHIMOE REGISTRA BP: B6A5
VVEDITE SODERZHIMOE REGISTRA AX: 3CF0

Manual check:
B6A5h = B 6 A 5 = 1011 0110 1010 0101
3CF0h = 3 C F 0 = 0011 1100 1111 0000

Program output: (BP) = 1011011010100101 (AX) = 0011110011110000
```

Рисунок 4 - Проверка результата вывода

7. Вывод

В ходе выполнения лабораторной работы была разработана программа, вызываемая из DOS. Программа вводит с клавиатуры шестнадцатеричные значения для регистров BP и AX, преобразует введенные числа во внутреннее 16-битное представление и выводит их двоичное содержимое на экран. Ручная проверка подтвердила правильность результата.